Py

Day 10/55 - Tuples in Python

Ordered, immutable collections for fixed data

Prepared by Satyanarayan Sen

Video Summary

This Telusko lesson introduces Python tuples as ordered collections for storing multiple values. It covers tuple creation, immutability, supported functions, unpacking, nested mutable objects, and membership checks with the in keyword.

Source video: https://www.youtube.com/watch?v=TQqDBeHq7IU&list=PLsyeobzWxl7omDoEYrrf3oXvXxa6MPgek&index=9

1 Defining Tuples

Tuples can be written with round brackets, or with comma-separated values. The comma is what really creates the tuple.

() Round brackets

Use () when you want the tuple shape to be obvious.

, Comma values

Values separated by commas can also create a tuple.

1, One item

A single-item tuple needs a trailing comma.

```
Tuple creation examples
>>> nums = (25, 12, 36, 95, 14)
>>> names = "navin", "kiran", "harsh"
>>> one = (5,)
>>> type(names)
<class 'tuple'>
```

2 Immutability

Tuples are immutable. After a tuple is created, you cannot assign a new value to an index, insert an item, or remove an item from it.

```
Immutable tuple example
>>> nums = (25, 12, 36, 95, 14)
>>> nums[1] = 77
TypeError: 'tuple' object does not support item assignment
>>> nums
(25, 12, 36, 95, 14)
```

Tuple rule

Use tuples when the order matters and the collection should not be changed after creation.

3 Supported Operations

Tuples support useful built-in functions. Their own methods are limited, mainly count() and index().

Operation	Purpose	Example result
len(nums)	Count items.	5
min/max/sum	Work on numeric tuples.	12, 95, 182
count(x)	Count matching values.	nums.count(12) -> 1
index(x)	Find first matching index.	nums.index(36) -> 2

4

Tuple Unpacking

Unpacking lets you assign tuple values directly to individual variables. The number of variables should match the number of values.

```
Unpacking examples
>>> student = (101, "Kiran", 89)
>>> roll, name, marks = student
>>> roll
101
>>> name
'Kiran'
>>> marks
89
```

=

Readable code

Unpacking gives clear names to tuple positions.

3

Exact match

Three tuple values need three receiving variables.

rec

Fixed records

Tuples are helpful for records like id, name, marks.

Unpacking rule

If the counts do not match, Python raises an error. Keep the tuple shape and variable count aligned.

5

Handling Nested Objects

The tuple itself cannot be changed, but if it contains a mutable object such as a list, the inside of that list can still change.

```
Nested list inside tuple
>>> data = (10, [20, 30], 40)
>>> data[1].append(99)
>>> data
(10, [20, 30, 99], 40)
>>> data[1] = [1, 2]
TypeError: 'tuple' object does not support item assignment
```

Important difference

You cannot replace the list stored inside the tuple, but you can mutate the list object itself.

slot **Tuple slot**

The tuple position still points to the same list object.

list **List object**

The list inside the tuple can grow or change.

err **No replace**

Assigning a new object to a tuple index fails.

6 Searching with in

The in keyword checks whether a value exists inside a tuple and returns True or False.

```
Membership examples
>>> names = ("navin", "kiran", "harsh")
>>> "kiran" in names
True
>>> "rahul" in names
False
```

Search rule

Use in when you need a direct yes-or-no membership check before using a tuple value.

7 Quick Practice

```
Practice example
>>> point = (4, 7)
>>> x, y = point
>>> len(point)
2
>>> 7 in point
True
```

8

Tuple Recap

Tuples are close to lists in how they store ordered data, but their immutability makes them useful for fixed values and safer records.

Feature	Tuple	List
Syntax	(10, 20) or 10, 20	[10, 20]
Mutability	Cannot change tuple slots.	Can change, insert, remove.
Methods	count(), index()	Many methods for editing.
Best use	Fixed records and constants.	Changing collections.

Beginner rule

Choose a tuple when ordered values should stay stable. Choose a list when values need to change.

Day 10/55 takeaway

Tuples are ordered and immutable. Use them for fixed records, unpacking, searching, and safe collection-style data.